Podem utilitzar una imatge buida ("*scratch*") per construir imatges des de zero. No és la millor opció, possiblement és molt millor usar una imatge *slim*, o bé fer un *commit* des d'un contenidor actiu.

La imatge *scratch* està buida, per tant, no hi ha llibreries… això fa que els arxius binaris no puguin usar llibreries de forma dinàmica. Les compilacions han de ser estàtiques!

Pràctica 1:

- Creem el codi font a *hw.c*

```
#include <stdio.h>

int main(){
      printf("Hooola mon!\n");
      return 0;
}
```

- El compilem de forma estàtica

```
gcc -static -o hw hw.c
```

- Comprovem que funcioni

```
chmod +x hw
./hw
```

- Creem l'arxiu Dockerfile

```
FROM scratch
COPY hw /
CMD ["/hw"]
```

- Creem la imatge

```
docker build -t hola:latest .
```

- Arranquem el contenidor

```
docker run --name prova hola:latest
```

- Eliminem contenidor i imatge

```
docker stop prova
docker rmi -f hola:latest
```

Expliqueu cadascuna de les comandes utilitzades i els seus paràmetres. Adjunteu captures de pantalla on es comprovi l'execució de cada comanda i el funcionament de tota la pràctica.

Pràctica 2:

Ara crearem una imatge a partir d'una imatge *slim* de Python.

- Creem l'arxiu de codi app.py

```
import os
import socket
from flask import Flask
from redis import Redis, RedisError

# Connect to Redis
redis = Redis(
    host=os.getenv("REDIS_HOST", "localhost"),
    port=6379,
    db=0,
    socket_connect_timeout=2,
    socket_timeout=2,
)

app = Flask(__name__)


@app.route("/")
def hello():
    try:
        visits = redis.incr("counter")
    except RedisError:
        visits = "<i>cannot connect to Redis, counter disabled</i>"

    html = (
        "<h3>Hoooola {name}!</h3>"
        "<b>Hostname:</b> {hostname}<br/>"
        "<b>Visits:</b> {visits}"
    )
    return html.format(
        name=os.getenv("NAME", "world"),
        hostname=socket.gethostname(),
        visits=visits,
    )


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=80)
```

- Creem el fitxer *requirements.txt*

```
Flask
Redis
```

- Creem el Dockerfile

```
FROM python:3.10-slim

RUN apt-get update && apt-get install -y \
    redis-server \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 80

CMD ["sh", "-c", "redis-server --daemonize yes && python app.py"]
```

- Creem la imatge

```
docker build -t holahtml .
```

- Arranquem el contenidor

```
docker run --rm --name prova2 -p 4000:80 holahtml
```

- Provem al navegador → http://localhost:4000

Expliqueu cadascuna línia a línia l'arxiu Dockerfile. Adjunteu captures de pantalla on es comprovi l'execució de cada comanda i el funcionament de tota la pràctica.